index into that array from another thread, you can use `std::kill_dependency()` to let the compiler know that it doesn't need to reread the contents of the array entry, as in the following example:

```
int global_data[]={ ... };
std::atomic<int> index;
void f()
{
    int i=index.load(std::memory_order_consume);
    do_something_with(global_data[std::kill_dependency(i)]);
}
```

Of course, you wouldn't normally use `std::memory_order_consume` at all in such a simple scenario, but you might call on `std::kill_dependency()` in a similar situation with more complex code. You must remember that this is an optimization, so it should only be used with care and where profiling has demonstrated the need.

Now that I've covered the basics of the memory orderings, it's time to look at the more complex parts of the synchronizes-with relation, which manifest in the form of *release sequences*.

5.3.4. Release sequences and synchronizes-with

Back in section 5.3.1, I mentioned that you could get a synchronizes-with relationship between a store to an atomic variable and a load of that atomic variable from another thread, even when there's a sequence of read-modify-write operations between the store and the load, provided all the operations are suitably tagged. Now that I've covered the possible memory-ordering "tags," I can elaborate on this. If the store is tagged with `memory_order_release`, `memory_order_acq_rel`, or `memory_order_seq_cst`, and the load is tagged with `memory_order_consume`, `memory_order_acquire`, or `memory_order_seq_cst`, and each operation in the chain loads the value written by the previous operation, then the chain of operations constitutes a *release sequence* and the initial store synchronizes-with (for `memory_order_acquire` or `memory_order_seq_cst`) or is dependency-ordered-before (for `memory_order_consume`) the final load. Any atomic read-modify-write operations in the chain can have *any* memory ordering (even `memory_order_relaxed`).

Listing 5.11. Reading values from a queue with atomic operations